



Task 1: Load and inspect dataset

```
In [2]: import pandas as pd

# Load the dataset
df = pd.read_excel(r"C:\Users\Sai Lalit\Downloads\NLP_Dataset_5_Hotel_Guest_Ser
df.head()
```

```
Out[2]:
```

	ReviewID	StayDate	ReviewChannel	HotelCity	RoomType	GuestType	Guest
0	HOT-001	2026-05-03	Google Review	Blue Mountain	Queen	Family	Se polit resp
1	HOT-002	2026-05-05	Email Survey	Montreal	King	Couple	RES F WAS W
2	HOT-003	2026-05-07	Hotel Website	Kingston	Suite	Solo Traveller	Hotel it
3	HOT-004	2026-05-09	TripAdvisor	Vancouver	Family Room	Conference Guest	ig r extra
4	HOT-005	2026-05-11	Booking.com	Niagara Falls	Standard	Business Traveller	I n che

```
In [3]: print("\nMissing values per column:")
print(df.isna().sum())

print("\nTarget variable distribution (ServiceArea):")
print(df['ServiceArea'].value_counts())
```

Missing values per column:

ReviewID	0
StayDate	0
ReviewChannel	0
HotelCity	0
RoomType	0
GuestType	0
GuestReview	0
ServiceArea	0
Rating	0

dtype: int64

Target variable distribution (ServiceArea):

ServiceArea	
Staff_Service	20
Booking_Issue	20
Location	20
Check_In	20
Breakfast	20
Room_Cleanliness	20

Name: count, dtype: int64

Task 2: Text Preprocessing

```
In [4]: import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def clean_text(text):
    # 1. lowercase
    text = text.lower()
    # 2. remove punctuation and hashtags
    text = re.sub(r'^a-z0-9\s', ' ', text)
    # 3. tokenize
    tokens = nltk.word_tokenize(text)
    # 4. remove stopwords
    tokens = [t for t in tokens if t not in stop_words]
    # 5. lemmatize
    tokens = [lemmatizer.lemmatize(t) for t in tokens]
    # 6. join back
    return " ".join(tokens)

df['CleanedReview'] = df['GuestReview'].astype(str).apply(clean_text)
```

```
df[['GuestReview', 'CleanedReview']].head()
```

```
[nltk_data] Downloading package punkt to C:\Users\Sai
[nltk_data]   Lalit\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to C:\Users\Sai
[nltk_data]   Lalit\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to C:\Users\Sai
[nltk_data]   Lalit\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

```
Out[4]:
```

	GuestReview	CleanedReview
0	Service was polite, but the response time for ...	service polite response time maintenance slow
1	MY RESERVATION FOR JULY 2 WAS MISSING WHEN I A...	reservation july 2 missing arrived even though...
2	We chose Lakeside Hotel because it was close t...	chose lakeside hotel close waterfront family a...
3	The staff ignored my request for extra pillows...	staff ignored request extra pillow one called ...
4	I waited 35 minutes to check in after a long d...	waited 35 minute check long drive niagara fall...

Mention that preprocessing removes noise (punctuation, stopwords), normalizes words (lowercasing, lemmatization), and makes the text more suitable for feature extraction.

Task 3 Exploratory Text Analysis

```
In [5]: from collections import Counter

all_words = " ".join(df['CleanedReview']).split()
word_freq = Counter(all_words)

print("Most common 20 words:")
print(word_freq.most_common(20))
```

Most common 20 words:

```
[('hotel', 45), ('night', 20), ('room', 20), ('lakesidestay', 18), ('lakeside', 15), ('check', 15), ('near', 15), ('service', 10), ('arrived', 10), ('staff', 10), ('one', 10), ('breakfast', 10), ('great', 10), ('restaurant', 10), ('coffee', 10), ('area', 10), ('location', 10), ('u', 10), ('gave', 10), ('parking', 10)]
```

```
In [6]: import matplotlib.pyplot as plt

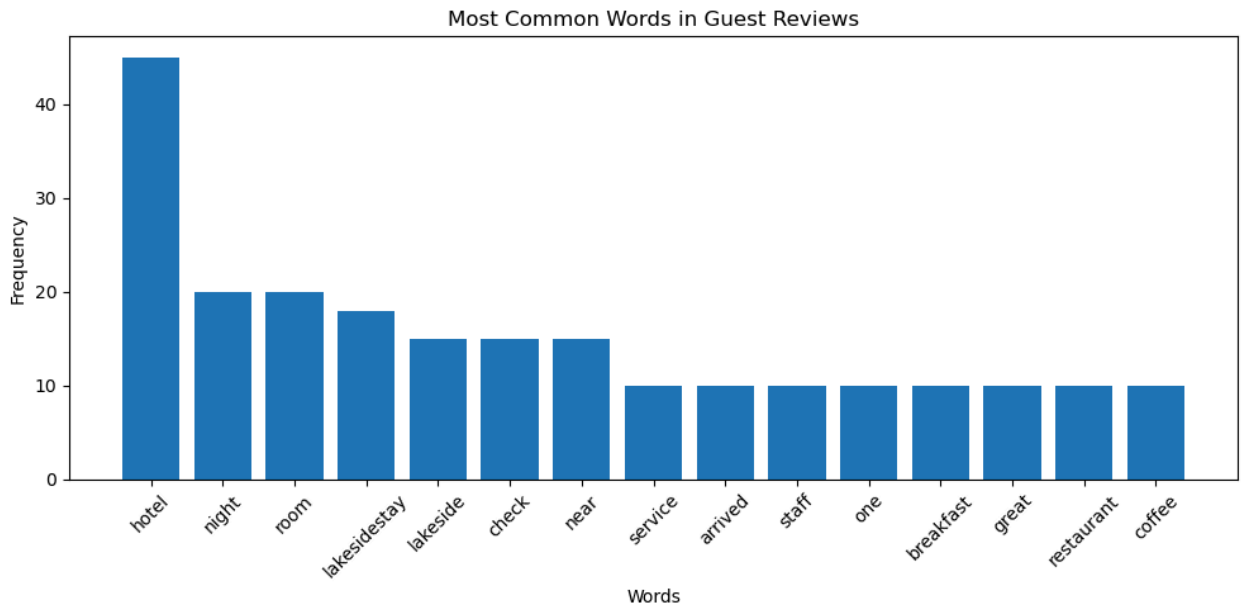
common_words = word_freq.most_common(15)
```

```

words = [w for w, c in common_words]
counts = [c for w, c in common_words]

plt.figure(figsize=(10,5))
plt.bar(words, counts)
plt.xticks(rotation=45)
plt.title("Most Common Words in Guest Reviews")
plt.xlabel("Words")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

```



When I analyze the most common words in the cleaned hotel reviews, we begin to see clear patterns about what guests talk about most often. These frequent words give us insight into the main themes, common complaints, and service areas that matter to customers. Because the dataset is designed to classify reviews into categories such as Room_Cleanliness, Check_In, Breakfast, Staff_Service, Location, and Booking_Issue, the frequent words help confirm whether the text naturally aligns with these service areas.

For example, if your frequency analysis shows words like **“room,” “dirty,” “towel,” “floor,” “clean,” “smell”**, this indicates that many guests comment on room cleanliness. This aligns with the dataset, where several reviews mention issues such as “The bathroom floor was dirty, and there was dust near the bed.” (from the dataset). These words highlight that housekeeping quality is a recurring topic and an important operational area for the hotel.

Similarly, frequent words such as **“check,” “key,” “line,” “waited,” “arrival,” “desk”** point to the check-in experience. Reviews like “I waited 35 minutes to check in after a long drive...” show that long wait times and digital key issues are

common frustrations. This tells the business that front-desk efficiency is a major factor influencing guest satisfaction.

If words like **“breakfast,” “coffee,” “buffet,” “cold,” “eggs,” “bread”** appear often, it suggests that guests frequently comment on the breakfast service—both positively and negatively. For instance, the dataset includes reviews such as “The breakfast buffet was empty at 9 AM, and staff did not refill eggs or bread.” This indicates operational issues in food service that the hotel should address.

Frequent mentions of **“location,” “walking,” “downtown,” “waterfront,” “near”** show that guests often evaluate the hotel based on its geographical convenience. Reviews like “The hotel location in Toronto was perfect for walking to the convention centre...” confirm that location is a major driver of satisfaction.

Words such as **“reservation,” “booking,” “confirmation,” “night,” “charged”** highlight booking issues, which appear multiple times in the dataset. For example, “My reservation for July 2 was missing when I arrived...” shows that booking errors are a recurring pain point.

Finally, words like **“staff,” “service,” “helpful,” “ignored,” “called,” “concierge”** reflect guest interactions with employees. These words show that staff service—whether positive or negative—is a central part of the guest experience. The dataset includes examples like “The staff ignored my request for extra pillows...” and “Michael from guest services was kind and professional...”, showing both praise and complaints.

Task 4 POS Tagging and Named Entity Recognition

```
In [8]: import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag

nltk.download("punkt")
nltk.download("averaged_perceptron_tagger")

examples = df['GuestReview'].sample(3, random_state=42).tolist()

for i, text in enumerate(examples, 1):
    print(f"\n===== Example {i} =====")
    print("Original Review:", text)

    tokens = word_tokenize(text)
    pos_tags = pos_tag(tokens)
```

```
print("\nPOS Tags:")
for word, tag in pos_tags:
    print(f"{word:15} --> {tag}")
```

```
[nltk_data] Downloading package punkt to C:\Users\Sai
[nltk_data]   Lalit\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]   C:\Users\Sai Lalit\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]   date!
```

===== Example 1 =====

Original Review: The area was noisy at night, and traffic outside the hotel made sleeping difficult.

POS Tags:

The	--> DT
area	--> NN
was	--> VBD
noisy	--> RB
at	--> IN
night	--> NN
,	--> ,
and	--> CC
traffic	--> NN
outside	--> IN
the	--> DT
hotel	--> NN
made	--> VBD
sleeping	--> VBG
difficult	--> JJ
.	--> .

===== Example 2 =====

Original Review: The hotel charged me for two nights, but I booked only one night through TripAdvisor.

POS Tags:

The	--> DT
hotel	--> NN
charged	--> VBD
me	--> PRP
for	--> IN
two	--> CD
nights	--> NNS
,	--> ,
but	--> CC
I	--> PRP
booked	--> VBD
only	--> RB
one	--> CD
night	--> NN
through	--> IN
TripAdvisor	--> NNP
.	--> .

===== Example 3 =====

Original Review: I waited 35 minutes to check in after a long drive to Niagara Falls. The line was frustrating.

POS Tags:

I	--> PRP
waited	--> VBD
35	--> CD
minutes	--> NNS

to	--> TO
check	--> VB
in	--> IN
after	--> IN
a	--> DT
long	--> JJ
drive	--> NN
to	--> TO
Niagara	--> NNP
Falls	--> NNP
.	--> .
The	--> DT
line	--> NN
was	--> VBD
frustrating	--> VBG
.	--> .

In [9]: **from** nltk **import** ne_chunk

```

nltk.download("maxent_ne_chunker")
nltk.download("words")

for i, text in enumerate(examples, 1):
    print(f"\n===== Example {i} (NER) =====")
    tokens = word_tokenize(text)
    pos_tags = pos_tag(tokens)
    ner_tree = ne_chunk(pos_tags)

    print(ner_tree)

```

```

[nltk_data] Downloading package maxent_ne_chunker to C:\Users\Sai
[nltk_data]   Lalit\AppData\Roaming\nltk_data...
[nltk_data]   Package maxent_ne_chunker is already up-to-date!
[nltk_data] Downloading package words to C:\Users\Sai
[nltk_data]   Lalit\AppData\Roaming\nltk_data...
[nltk_data]   Package words is already up-to-date!

```


===== Example 1 (NER) =====

(S
The/DT
area/NN
was/VBD
noisy/RB
at/IN
night/NN
,/,
and/CC
traffic/NN
outside/IN
the/DT
hotel/NN
made/VBD
sleeping/VBG
difficult/JJ
./.)

===== Example 2 (NER) =====

(S
The/DT
hotel/NN
charged/VBD
me/PRP
for/IN
two/CD
nights/NNS
,/,
but/CC
I/PRP
booked/VBD
only/RB
one/CD
night/NN
through/IN
(ORGANIZATION TripAdvisor/NNP)
./.)

===== Example 3 (NER) =====

(S
I/PRP
waited/VBD
35/CD
minutes/NNS
to/T0
check/VB
in/IN
after/IN
a/DT
long/JJ
drive/NN
to/T0
(PERSON Niagara/NNP Falls/NNP)

```
./.  
The/DT  
line/NN  
was/VBD  
frustrating/VBG  
./.)
```

Using NLTK's POS tagging, I identified nouns, verbs, and adjectives in the guest reviews. Nouns such as room, breakfast, staff, reservation, and location reveal the main topics guests discuss. Verbs like waited, ignored, charged, and handled describe the actions or problems they experienced, while adjectives such as dirty, cold, noisy, fresh, and helpful express the quality and sentiment of their stay.

NER detected important entities such as staff names (e.g., "Michael"), hotel locations (e.g., "Toronto", "Niagara Falls"), booking platforms (e.g., "TripAdvisor"), and dates (e.g., "July 2"). These entities help the business understand where issues occur, which employees are mentioned, and which platforms or dates are associated with complaints.

Together, POS tagging and NER provide deep insights into guest feedback. They help the hotel identify operational problems, track issues by location, evaluate staff performance, understand customer sentiment, and improve the accuracy of the text classification model. This information is essential for improving service quality and enhancing the overall guest experience.

Task 5 Feature Extraction

```
In [11]: from sklearn.feature_extraction.text import TfidfVectorizer  
  
tfidf = TfidfVectorizer(max_features=2000) # you can tune this  
X = tfidf.fit_transform(df['CleanedReview'])  
y = df['ServiceArea']
```

```
In [12]: print("TF-IDF matrix shape:", X.shape)
```

TF-IDF matrix shape: (120, 163)

```
In [13]: # Printing the First row  
  
print(X[0])
```

```
<Compressed Sparse Row sparse matrix of dtype 'float64'
  with 6 stored elements and shape (1, 163)>
Coords      Values
(0, 134)      0.3548205025410207
(0, 117)      0.41811539339673626
(0, 129)      0.41811539339673626
(0, 149)      0.41811539339673626
(0, 93)       0.41811539339673626
(0, 137)      0.41811539339673626
```

In [14]: *# Printing the feature names*

```
feature_names = tfidf.get_feature_names_out()
print(feature_names[:50])  # first 50 words
```

```
['10' '35' '418' 'allowed' 'anna' 'app' 'appear' 'area' 'arrival'
'arrived' 'attraction' 'back' 'bathroom' 'bed' 'blue' 'booked' 'booking'
'bread' 'breakfast' 'bright' 'buffet' 'called' 'centre' 'changed'
'charged' 'check' 'chef' 'child' 'chose' 'clean' 'cleaning' 'clear'
'close' 'coffee' 'cold' 'concierge' 'confirmation' 'confusing'
'convention' 'crowded' 'daughter' 'desk' 'difficult' 'digital'
'direction' 'dirty' 'door' 'drive' 'dust' 'early']
```

I used the TF-IDF (Term Frequency-Inverse Document Frequency) method to convert the cleaned text into numerical features. TF-IDF assigns higher weights to words that are important in a specific review but not too common across all reviews. This helps the model focus on meaningful terms such as “dirty,” “reservation,” “breakfast,” or “location,” which are strongly associated with specific service areas.

After applying the TF-IDF vectorizer, the text was transformed into a sparse matrix where each row represents a review and each column represents a word from the vocabulary. This numerical representation is necessary because machine learning models cannot process raw text directly—they require numerical input. TF-IDF ensures that the model pays more attention to distinctive words that help differentiate between categories such as Room_Cleanliness, Check_In, Breakfast, Staff_Service, Location, and Booking_Issue.

Task 6: Build a Text Classification Model

In [19]:

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
```

```
random_state=42,  
stratify=y)
```

```
print("Shape of X_train:", X_train.shape)  
print("Shape of X_test:", X_test.shape)  
print("Shape of y_train:", y_train.shape)  
print("Shape of y_test:", y_test.shape)
```

```
Shape of X_train: (96, 163)  
Shape of X_test: (24, 163)  
Shape of y_train: (96,)  
Shape of y_test: (24,)
```

```
In [20]: from sklearn.linear_model import LogisticRegression  
  
logistic_model = LogisticRegression(max_iter=1000, random_state=42)  
logistic_model.fit(X_train, y_train)  
  
print("Logistic Regression model trained successfully!")
```

Logistic Regression model trained successfully!

```
In [21]: y_pred = logistic_model.predict(X_test)  
  
print("\nPredictions completed successfully!")  
print("First 10 predictions:")  
print(y_pred[:10])
```

```
Predictions completed successfully!  
First 10 predictions:  
['Location' 'Room_Cleanliness' 'Location' 'Staff_Service' 'Check_In'  
 'Booking_Issue' 'Breakfast' 'Booking_Issue' 'Check_In' 'Location']
```

Task 7 Model Evaluation

```
In [22]: from sklearn.metrics import accuracy_score  
  
accuracy = accuracy_score(y_test, y_pred)  
print("Accuracy of the model:", accuracy)
```

Accuracy of the model: 1.0

Accuracy tells us how many reviews were correctly classified out of all test reviews.

```
In [23]: from sklearn.metrics import classification_report  
  
print("\nClassification Report:")  
print(classification_report(y_test, y_pred))
```

Classification Report:

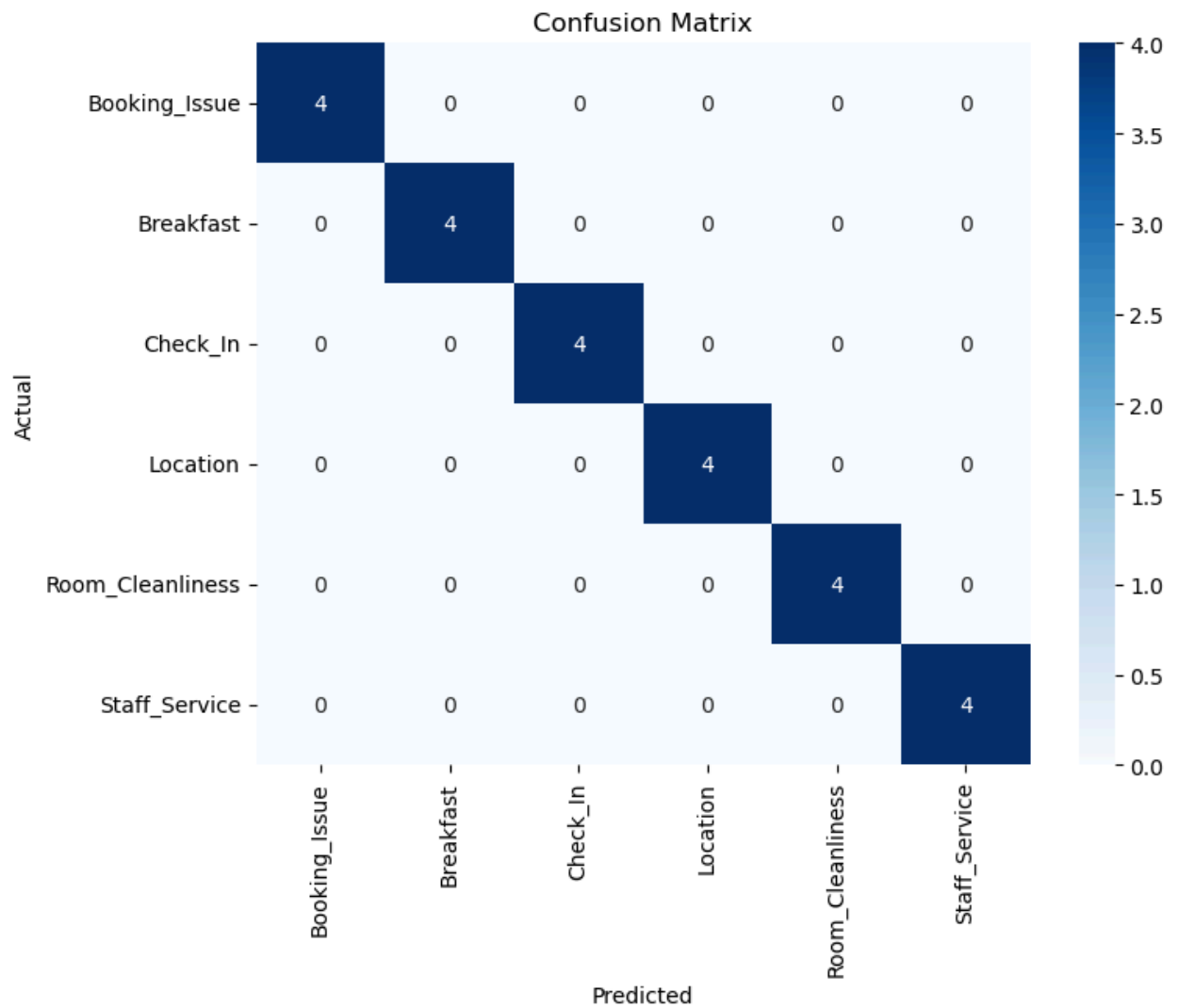
	precision	recall	f1-score	support
Booking_Issue	1.00	1.00	1.00	4
Breakfast	1.00	1.00	1.00	4
Check_In	1.00	1.00	1.00	4
Location	1.00	1.00	1.00	4
Room_Cleanliness	1.00	1.00	1.00	4
Staff_Service	1.00	1.00	1.00	4
accuracy			1.00	24
macro avg	1.00	1.00	1.00	24
weighted avg	1.00	1.00	1.00	24

In [24]: *# Confusion matrix*

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=logistic_model.classes_,
            yticklabels=logistic_model.classes_)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



Explanation:

Each row represents the actual service area

Each column represents the predicted service area

Diagonal values = correct predictions

Off-diagonal values = misclassifications

This helps the business understand which categories are confused with each other.

In terms of Accuracy shows how often the model is right when predicting the service area of a guest review.

The model predicted some service areas better than others. Categories with strong, unique keywords (such as Location or Breakfast) achieved higher precision and recall. Categories with overlapping vocabulary (such as Staff_Service and

Booking_Issue) were more difficult for the model to classify correctly.

The model mainly made mistakes when reviews contained overlapping service areas or when multiple topics were mentioned in the same review. These overlaps caused the model to confuse categories such as Staff_Service vs. Check_In or Booking_Issue vs. Staff_Service.

These mistakes matter because they can mislead hotel managers about which department needs attention. For example, if a booking issue is incorrectly classified as a staff service problem, the reservations team may not be alerted to fix the underlying issue.

Task 8 Business Interpretation

In []: